Programming Assignment 2: Support Vector Machines

Due: Friday 10/30/2026 @ 11:59pm EST

The purpose of programming assignments is to implement model(s) we discuss in lecture and to then perform experiments on them. We are primarily using the Python programming language in this version of the assignments. Your task this assignmnet is to implement the Support Vector Machine classifier from class using the Sequential Minimal Optimization algorithm and to build ensembles from SVMs to handle multiclass classification problems.

Task 1: Required `modules`

In order to build/run this assignment you will need to install some packages if you are not using the SCC:

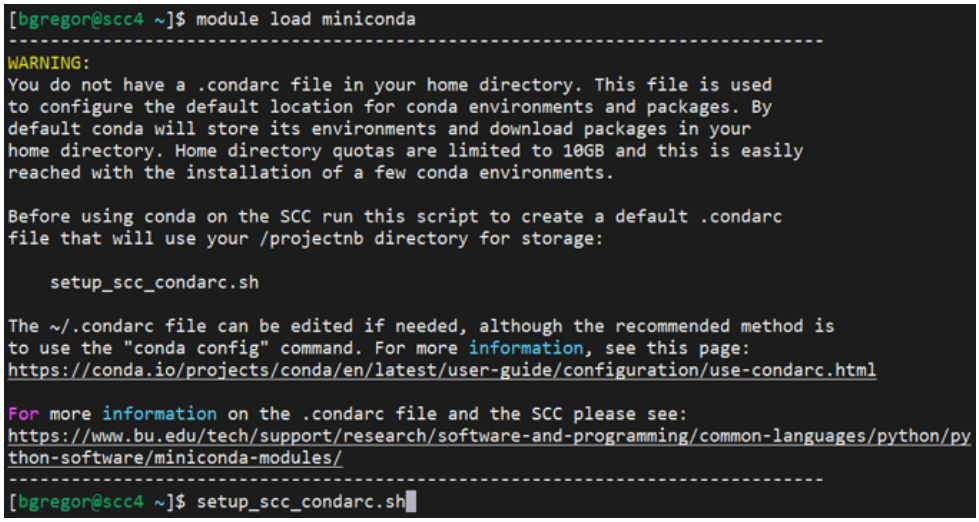
- If you are developing locally I have provided a `requirements.txt` file which you can use to install the required dependencies with the command:

  ```
  python3 -m pip install -r requirements.txt
  ```

- If you are using the SCC then you will need to load the correct modules after logging in. You can do so with the following terminal commands:

  ```
  module load miniconda
  ```

  This *may* cause a bunch of stuff to be printed out similar to the following image:



  If you see this message or something like that its ok! This just means you've never setup `miniconda` on the SCC before! This message is telling you that you need to run the following script from your SCC terminal:

  ```
  setup_scc_condarc.sh
  ```

  This script will perform some setup on your behalf. Something special to note about the SCC is that your home directory only has a 10GB quota, and its **very** easy to exhaust that storage if you let `miniconda` write to your home directory! If you run this script, it will configure `miniconda` to **not** use your home directory (which is good!).

If you **didn't** see this message, you might want to consider updating your ~/.condarc file to point to the cs542 directory on the SCC. If you didn't see this message then it is almost certain your ~/.condarc is telling miniconda to write to some **other** class's/project's directory and you want to switch it for this class!

Anyways, once miniconda is setup, you can load and activate the following modules to get into a nice python environment:

```
module load miniconda # you already did this
module load academic-ml
conda activate fall-2026-pyt
```

The environment fall-2026-pyt is for the fall semester of year 2026 specialized for pytorch. There are also tensorflow and jax versions but if we need a neural network library we'll be using pytorch. This environment will have everything we need for this class already installed!

Task 2. The Starter Code

Included with this pdf is a directory called `src`. Inside this directory is the programming assignment starter code, tests, and data. There is also a sample `train_svmc.py` script which should give you an idea of how to use this api to train some models.

I have implemented quite a bit for you. I have implemented several kernel functions: Linear Kernel (which corresponds to no feature warping at all), Quadradic Kernel (which corresponds to a finitely-sized feature warp), and the Radial Basis Function Kernel (RBF Kernel for short which corresponds to an infitely-sized feature warp). You can find these in the `src.svm.kernels` package. If you want to add more kernels you're welcome!. In the `src.svm.data` package you will find some code for loading data stored in the `c4.5` format as well as some datatypes useful for describing the features in such a dataset. There are also five datasets of various difficulties. I have included two "toy" dataset called the `play_outside` dataset (this is the dataset we've used in lecture) and the `small_linsep` dataset (which I engineered to have an easy-to-see optimal hyperplane). There are also `voting`, `spam`, and `volcanoes` datasets. Each of the "real" datasets has a `.info` file in their corresponding directory which describes the dataset. Feel free to read them! I have **not** included any multiclass datasets by default, you might want to add some to test your ensembles!

The meat of this assignment is in the `src.svm.clf` module (in the `src/svm/clf.py` file) as well as in the `src.svm.opt.smo` module (in the `src/svm/opt/smo.py` file). These files contains partial implementation of a support vector machine classifier, One-Vs-Rest (OVR) ensemble classification, One-Vs-One (OVO) ensemble classification, and the Sequential Minimal Optimization algorithm. This assignment is all about finishing these implementations, improving them, and then applying them!

I have also included some starter unit tests in the `tests` directory. These unit tests are not exhaustive so you will need to add more!

Task 3: Sequential Minimal Optimization (ugrads: required & 30 points, grads: required & 20 points)

In the file `src/svm/opt/smo.py` you will find the starter code for the `smo` algorithm which has a function called `smo`. You will need to finish this implementation in order to train a support vector machine. This implementation has five functions you will need to incomplete. Each of these functions has a docstring detailing what you need to do to complete them:

- function `_get_errors`. This function is where you will calculate the error terms used to detect whether alpha values satisfy the KKT conditions. These error values will also be used when updating a pair of alpha values.
- function `_alphas_violating_ktt`. This function will return the indices of any alpha values currently violating at least one KKT condition.
- function `_choose_alpha_violating_kkt`. This function will choose one index from the set of alphas violating kkt conditions. This chosen alpha will be the alpha you update in an iteration of the SMO algorithm. In this part of the code I want you to choose uniformly at random from all of the alpha values currently violating KKT conditions.
- function `_smo_step`. This function is where, given two alpha values α_1 and α_2 you will update both (if possible) such that α_1 hopefully no longer violates a KKT condition. See the docstring for more details.
- function `smo`. This function will use all of the other four functions to choose an alpha value that is currently violating a KKT condition, choose another alpha value to pair with it (see

the docstring for details), performs an update step and then checks for convergence (see the docstring for details). These steps will be repeated until convergence is reached or a maximum number of iterations has been exhausted without convergence.

I really recommend that you test your code! I have provided a starter unit test file that will test on the `play_outside` dataset with different kernels. I will note that the `small_linsep` dataset might be easier to use to debug and/or write your own tests on!

Task 4: SVM Classifier (ugrads: required & 20 points, grads: required & 10 points)

In the file `src/svm/clf.py` you will find the type `SVMC`. You will need to complete two methods here:

- method `_to_w`. This method takes the alpha values produced by smo and calculates the weights of the corresponding hyperplane. We derived the solution to calculate $\vec{w}^*$ in lecture when we solved $\nabla_{\vec{w}}\mathcal{L} = \vec{0}$.
- method `predict`. This method is where your model will perform inference. See the docstring for more details, but don't forget that your model should produce labels in the label space the user of your code is expecting!

Task 5: Experiments (ugrads: required & 50 points, grads: required & 25 points)

Write a script (probably similar to `train_svmc.py`) that will perform 5-fold stratified cross validation using a repeatable random seed of 12345. Include in your report answers to the following questions under a section called `SVM Experiments`:

- What is the average 5-fold stratified cross validation classification accuracy of a svm restricted to a value of $C = 1000$ on each of the three "real" datasets using a linear kernel? What is the variance?
- For the `volcanoes` and `spam` datasets, plot the average 5-fold stratified cross validation classification accuracy (including error bars) as the value of C is lowered from 1000 to 1 using a linear kernel. On the x-axis choose values of C to test so that there are at least 5 evenly-spaced points. Does the accuracy improve smoothly as the value of C decreases? Can you explain the pattern in the graph?
- Pick 3 different values of C. How do the average 5-fold stratified cross validation classification accuracies change if you change the kernel from a linear kernel to a quadradic kernel? How about the variances?
- Pick 3 different values of C. How do the average 5-fold stratified cross validation classification accuracies change if you change the kernel from a quadradic kernel to a RBF kernel? How about the variances?

Task 6: Ensembles (ugrads: bonus & 25 points, grads: required & 20 points)

Now complete the `OVRC` and `OVOC` types in the file `src/svm/clf.py`. You will need to consider how each model will train its constituent SVMs (and complete the `fit` method) as well as how the ensemble will use each constituent SVM during inference (and complete the `predict` method). See the docstrings for these types for more details.

Task 7: Ensemble Experiments (ugrads: bonus & 25 points, grads: required & 25 points)

Write a script (probably similar to `train_svmc.py`) that will perform 5-fold stratified cross validation using a repeatable random seed of 12345 on multiclass datasets built from the MNIST handwritten digits and the IRIS datasets. Include in your report answers to the following questions under a section called `Ensemble Experiments`:

- What is the average 5-fold stratified cross validation classification accuracy of OVR and OVO ensembles restricted to a value of $C = 1000$ on the two multiclass datasets using a linear kernel? What is the variance? Does either type of ensemble seem to perform better? On which datasets?
- For the two multiclass datasets, plot the average 5-fold stratified cross validation classification accuracy (including error bars) as the value of C is lowered from 1000 to 1 using a linear kernel for each type of ensemble. On the x-axis choose values of C to test so that there are at least 5 evenly-spaced points. Does the accuracy improve smoothly as the value of C decreases? Can you explain the pattern in the graph? Does either type of ensemble seem to perform better? On which datasets?
- Pick 3 different values of C. How do the average 5-fold stratified cross validation classification accuracies change if you change the kernel from a linear kernel to a quadradic kernel for each ensemble type? How about the variances? Does either type of ensemble seem to perform better? On which datasets?
- Pick 3 different values of C. How do the average 5-fold stratified cross validation classification accuracies change if you change the kernel from a quadradic kernel to a RBF kernel for each ensemble type? How about the variances? Does either type of ensemble seem to perform better? On which datasets?

Task 8: SMO Tuning and Experiments (ugrads: no credit, grads: bonus & 50 points)

In the file `src/svm/opt/smo.py` you will find the function `_choose_alpha_violating_kkt`. In the intial version of your code you chose uniformly at random from the alpha values currently violating KKT conditions. Now I want you to design three separate heuristic schemes that attempt to guess the most "promising" alpha and choose that alpha to update.

Write a script (probably similar to `train_svmc.py`) that will perform 5-fold stratified cross validation using a repeatable random seed of 12345 on a single support vector machine. Include in your report answers to the following questions under a section called `SMO Heuristic Experiments`:

- Describe each of the three heuristic schemes you implemented. What decisions did you have to make and why did you design your heuristics the way you did?
- What is the average 5-fold stratified cross validation classification accuracy of a svm restricted to a value of $C = 1000$ on each of the three "real" datasets using a linear kernel for each of your heuristics? What is the variance? Also include a plot showing the number of iterations your SMO algorithm performed as a function of each of your three heuristics for the 5-fold stratified cross validation experiments. Include error bars for the number of iterations.
- For the `volcanoes` and `spam` datasets, plot the average 5-fold stratified cross validation classification accuracy (including error bars) as the value of C is lowered from 1000 to 1 using a linear kernel for each of your heuristics. On the x-axis choose values of C to test so that there are at least 5 evenly-spaced points. Does the accuracy improve smoothly as the value of C decreases? Can you explain the pattern in the graph? Which heuristics performed well and which didn't? Also include a plot of the number of iterations (including variances) for each of your heuristics across the 5-fold cross validation experiment where you were changing C.
- Pick 3 different values of C. How do the average 5-fold stratified cross validation classification accuracies change if you change the kernel from a linear kernel to a quadradic kernel for each of your heuristics? How about the variances? Also include a plot of the number of iterations as described earlier. Which heuristics seemed to converge quicker if convergence happened at all?
- Pick 3 different values of C. How do the average 5-fold stratified cross validation classification accuracies change if you change the kernel from a quadradic kernel to a RBF kernel? How about the variances? Also include a plot of the number of iterations as described earlier. Which heuristics seemed to converge quicker if convergence happened at all?

Write a script (probably similar to `train_svmc.py`) that will perform 5-fold stratified cross validation using a repeatable random seed of 12345 on multiclass datasets built from the MNIST handwritten digits and the IRIS datasets. Include in your report answers to the following questions under a section called `Ensemble Heuristic Experiments`:

- What is the average 5-fold stratified cross validation classification accuracy of OVR and OVO ensembles restricted to a value of $C = 1000$ on the two multiclass datasets using a linear kernel? What is the variance? Does either type of ensemble seem to perform better? On which datasets?
- For the two multiclass datasets, plot the average 5-fold stratified cross validation classification accuracy (including error bars) as the value of C is lowered from 1000 to 1 using a linear kernel for each type of ensemble and for each of your three heuristics. On the x-axis choose values of C to test so that there are at least 5 evenly-spaced points. Does the accuracy improve smoothly as the value of C decreases? Can you explain the pattern in the graph? Does either type of ensemble seem to perform better? On which datasets? Also include plots of the number of iterations each heuristic took across the different experiments. Did some heuristics perform well for different ensemble types? On which data?

- Pick 3 different values of C. How do the average 5-fold stratified cross validation classification accuracies change if you change the kernel from a linear kernel to a quadradic kernel for each ensemble type and for each of your three heuristics? How about the variances? Does either type of ensemble seem to perform better? On which datasets? Also include plots of the number of iterations each heuristic took across the different experiments. Did some heuristics perform well for different ensemble types? On which data?

- Pick 3 different values of C. How do the average 5-fold stratified cross validation classification accuracies change if you change the kernel from a quadradic kernel to a RBF kernel for each ensemble type and for each of your three heuristics? How about the variances? Does either type of ensemble seem to perform better? On which datasets? Also include plots of the number of iterations each heuristic took across the different experiments. Did some heuristics perform well for different ensemble types? On which data?

Task 9: Submitting Your Assignment

Please turn in **only your source code** into gradescope as well as a written report. If you turn in any of the other files I **will not use them**. For the code autograders, I will put my own copies of the standard library and will be using **only** your `clf.py` and `smo.py` files. Depending on how long the code autograders run for I may have to make multiple code autograders so be sure to submit to them all!